

Projeto Final da Disciplina

API de Otimização para o Problema de Corte Unidimensional (Optimization-as-a-Service)

Laboratório de Otimização — Ciências de Dados
Universidade Federal do Ceará

2026.1

1 Tema

Desenvolvimento, teste, documentação e publicação online de um serviço de API RESTful voltado para a resolução do **Problema de Corte Unidimensional** (One-Dimensional Cutting Stock Problem), integrando solvers de otimização (**Google OR-Tools**) com comunicação web moderna.

2 O Problema de Otimização: Corte Unidimensional

O Problema de Corte Unidimensional é um clássico de Pesquisa Operacional aplicado em indústrias para cortar estoques padrão de comprimento L de forma a atender uma demanda de subpeças de comprimentos menores l_i minimizando o desperdício total de material.

2.1 Cenário de Negócio

Uma empresa possui barras padrão de comprimento útil L (ex: barras de aço de 6m) e deve produzir itens menores com comprimentos l_i em quantidades especificadas pela demanda d_i . O objetivo é planejar os padrões de corte para satisfazer a demanda minimizando a quantidade de barras padrão cortadas.

2.2 Formulação Matemática (Kantorovich)

Conjuntos e Índices:

- $i \in \{1, \dots, m\}$: Índice dos itens de demanda solicitados pelos clientes.
- $j \in \{1, \dots, N\}$: Índice das barras padrão de estoque disponíveis. Onde N é um limite superior do número máximo de barras que seriam necessárias (ex: a soma de todas as demandas $\sum_i d_i$).

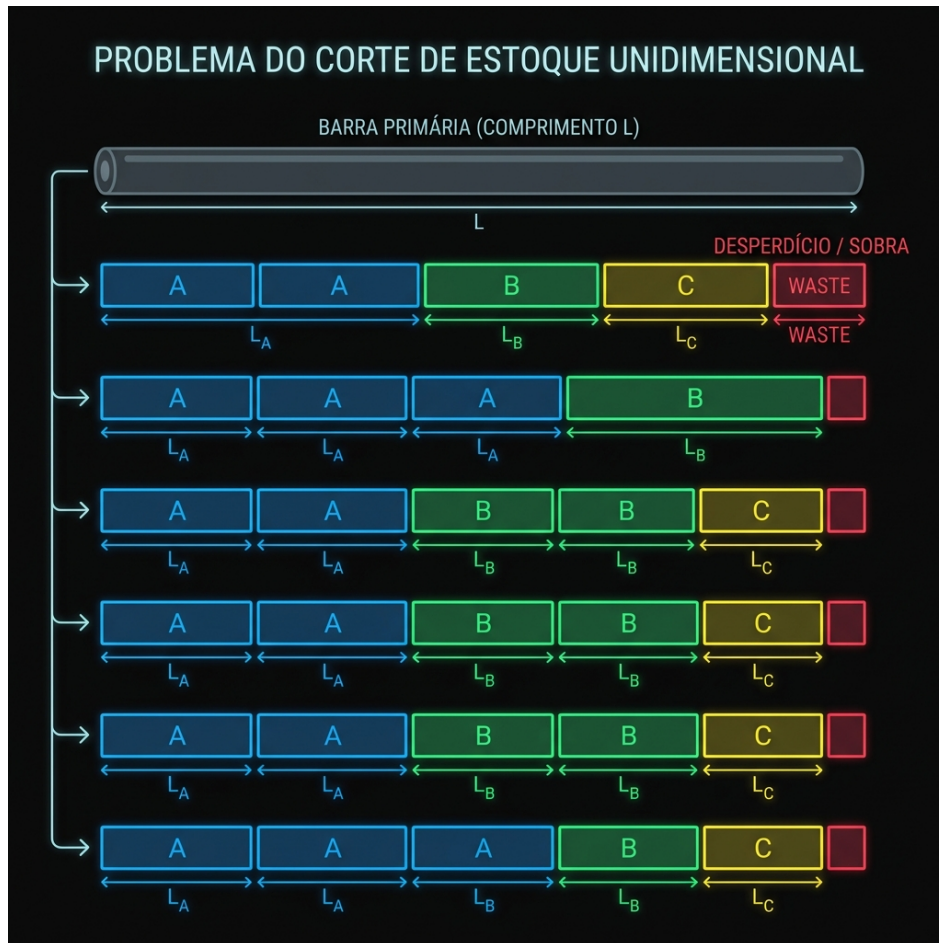


Figure 1: Esquema gráfico do corte unidimensional e desperdício associado.

Parâmetros:

- L : Comprimento útil de cada barra padrão em estoque.
- l_i : Comprimento necessário do item i .
- d_i : Demanda total exigida do item i .

Variáveis de Decisão:

- $y_j \in \{0, 1\}$: Indica se a barra j foi cortada/utilizada ($y_j = 1$) ou não ($y_j = 0$).
- $x_{ij} \in \mathbb{Z}_{\geq 0}$: Representa a quantidade de vezes que o item i é cortado na barra padrão j .

Modelo Matemático:

$$\text{Minimizar } Z = \sum_{j=1}^N y_j \quad (1)$$

$$\text{sujeito a: } \sum_{j=1}^N x_{ij} \geq d_i, \quad \forall i \in \{1, \dots, m\} \quad (2)$$

$$\sum_{i=1}^m l_i x_{ij} \leq L y_j, \quad \forall j \in \{1, \dots, N\} \quad (3)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, \dots, N\} \quad (4)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, N\} \quad (5)$$

A restrição (2) garante que toda a demanda seja suprida, e a restrição (3) garante que o comprimento total dos cortes em cada barra não exceda o tamanho limite L .

3 Requisitos Obrigatórios

3.1 Arquitetura e Ambiente

O repositório deve estar organizado de forma modular, contendo dependências isoladas em um ambiente virtual (`.venv`), um arquivo `requirements.txt` e a seguinte estrutura mínima:

```
projeto_corte_api/
  src/
    api.py           # Configuração do FastAPI e endpoints
    schemas.py       # Validadores Pydantic de entrada e saída
    auth.py          # Lógica de controle de acesso (API Key ou JWT)
    solver.py        # Modelagem matemática no Google OR-Tools
    config.py        # Configurações e ambiente (.env)
  tests/
    test_solver.py   # Testes unitários do modelo matemático
    test_api.py      # Testes de integração das rotas
  .env               # Armazenamento de segredos locais
  .gitignore         # Filtros de arquivos ignorados no Git
  requirements.txt   # Dependências
  README.md          # Manual de instalação e formulação matemática
  api_requests.http  # Exemplos de requisições de demonstração
```

3.2 Requisitos Funcionais

- **Endpoint POST /otimizar:** Recebe o comprimento padrão da barra, a lista de itens desejados com suas quantidades e um parâmetro opcional de tempo limite de processamento (`time_limit`).

- **Resposta Estruturada:** Retorna o status de resolução do solver (ex: `OPTIMAL`, `FEASIBLE`), o número total de barras padrão utilizadas, o desperdício total em comprimento e a especificação detalhada de corte de cada barra.
- **Endpoint GET /health:** Verifica se a API está online e operando normalmente.

3.3 Validação com Pydantic

- O comprimento padrão L e a demanda d_i devem ser estritamente positivos.
- Nenhum item pode ter comprimento individual l_i maior que o comprimento padrão da barra L .
- Falhas de tipagem ou restrições de schema devem retornar `422 Unprocessable Entity`.

3.4 Segurança e Autenticação

Em concordância com os tópicos da disciplina (Semana 11), a rota de otimização precisa ser protegida contra requisições não autorizadas. O aluno deve escolher entre:

- **Opção A: API Key** com cabeçalho `X-API-Key` verificado contra segredos armazenados no `.env`. Acesso negado gera erro 403.
- **Opção B: JWT (JSON Web Token)** com endpoint de login na rota `/auth/login` fornecendo token temporário com expiração (30 min) a ser passado via cabeçalho `Authorization: Bearer <token>`. Acesso inválido gera erro 401.

3.5 Confiabilidade (Time Limit)

A API não deve aceitar execuções infinitas. O solver do OR-Tools deve ser configurado com limite de tempo máximo padrão (ex: 60s) ou customizável via requisição (ex: até 120s), retornando a melhor solução encontrada até o momento quando o tempo se esgotar.

4 Contratos de Integração (JSON)

4.1 POST /otimizar — Entrada

```
{
  "comprimento_padrao": 3000,
  "itens": [
    {
      "id": "item_A",
      "comprimento": 1150,
      "quantidade": 3
    },
  ],
}
```

```
{
  "id": "item_B",
  "comprimento": 800,
  "quantidade": 4
},
{
  "id": "item_C",
  "comprimento": 450,
  "quantidade": 5
}
]
```

4.2 POST /otimizar — Resposta (200 OK)

```
{
  "status_solver": "OPTIMAL",
  "tempo_execucao_segundos": 0.045,
  "barras_utilizadas": 4,
  "desperdicio_total_mm": 3100,
  "plano_corte": [
    {
      "barra_id": 1,
      "itens_cortados": [
        {"item_id": "item_A", "quantidade": 2}
      ],
      "comprimento_utilizado": 2300,
      "sobra": 700
    },
    {
      "barra_id": 2,
      "itens_cortados": [
        {"item_id": "item_A", "quantidade": 1},
        {"item_id": "item_B", "quantidade": 2}
      ],
      "comprimento_utilizado": 2750,
      "sobra": 250
    },
    {
      "barra_id": 3,
      "itens_cortados": [
        {"item_id": "item_B", "quantidade": 2},
        {"item_id": "item_C", "quantidade": 3}
      ],
      "comprimento_utilizado": 2950,
      "sobra": 50
    }
  ],
  {
```

```

    "barra_id": 4,
    "itens_cortados": [
        {"item_id": "item_C", "quantidade": 2}
    ],
    "comprimento_utilizado": 900,
    "sobra": 2100
}
]
}
```

5 Testes e Implantação

Suíte de Testes com pytest: Mínimo de **10 testes distintos** cobrindo casos unitários de otimização (testes simples com uma barra e instâncias clássicas com desperdícios conhecidos) e testes de rotas da API (validações Pydantic inválidas, limites de tempo e bloqueios de segurança).

Implantação Pública (Deploy): O serviço deve ser publicado em um provedor de nuvem (Render, Hugging Face Spaces, Railway ou Fly.io), e a URL pública correspondente com a rota interativa do Swagger (/docs) deve ser explicitada de forma visível no topo do arquivo README.md.

6 Critérios de Avaliação

Nota final de 0 a 10 pontos baseada na seguinte rubrica avaliativa:

Nota	Critério Avaliativo
1.0	Organização de pastas, controle de dependências, ambiente virtual e README completo.
1.0	Modelagem algébrica correta implementada de forma modular utilizando OR-Tools.
1.0	Construção RESTful da API (FastAPI) respeitando verbos HTTP e códigos de erro.
1.0	Validação rígida de parâmetros de entrada e exceções tratadas (Pydantic).
1.0	Mecanismos de confiabilidade e time limit para o solver de otimização.
1.0	Segurança funcional de autenticação de rota (API Key ou JWT).
1.0	Práticas de proteção ao versionar segredos (.env fora do Git) e ocultação de traceback.
1.0	Documentação Swagger nativa rica em detalhes, tags, e exemplos nos Schemas.
1.0	Testes automatizados robustos (pytest) cobrindo cálculo e integração da API.
1.0	Implantação e hospedagem online estável da API acessível via Internet.